Explicación detallada de RankGA

Documento técnico alineado con la implementación actual del repositorio

18 de abril de 2026

1. Propósito de este documento

Este documento explica con detalle el algoritmo implementado en el repositorio `RankGA`. No describe un algoritmo genérico inventado aparte del código, sino la versión concreta que hoy existe en las clases `rankga.RankGA`, `rankga.Population`, `rankga.Individual`, `rankga.Problem` y `rankga.Gene`.

La meta es dejar claro:

- qué problema resuelve el framework;
- cuál es la arquitectura conceptual del sistema;
- cómo opera cada fase de la evolución;
- qué ecuaciones usa realmente la implementación;
- qué información genera como salida;
- y cuáles son sus supuestos, limitaciones y detalles que pueden provocar mala interpretación si solo se mira el nombre de las clases.

Este texto es deliberadamente exhaustivo. La idea no es solo decir "qué hace", sino dejar explícito *por qué* está organizado así, qué convenciones adopta y dónde conviene tener cuidado.

2. Idea general del algoritmo

RankGA es un algoritmo genético con cuatro rasgos estructurales centrales:

1. **Selección por rango.** La probabilidad esperada de supervivencia o replicación de un individuo no depende directamente de su valor numérico de fitness, sino de su *posición ordenada* dentro de la población.
2. **Mutación dependiente del rango.** La intensidad de mutación no es uniforme en toda la población. Los individuos mejor rankeados reciben menos perturbación y los peor rankeados reciben más.
3. **Emparejamiento por rango para recombinación.** La recombinación no empareja individuos al azar. Después de ordenar la población por fitness, el algoritmo empareja individuos contiguos en ese orden. Por tanto, la estructura del rango no solo controla la selección y la mutación, sino también *quién se recombina con quién.*
4. **Separación entre motor y problema.** El motor evolutivo es genérico y delega en el problema concreto la forma de representar genes, construir individuos, evaluar fitness y definir el significado de la intensidad de mutación.

La orientación del framework es de **maximización**. Un fitness mayor siempre es mejor. Toda la lógica de ordenamiento y selección está escrita con esa convención.

3. Arquitectura del sistema

3.1. Componentes principales

El núcleo del framework se reparte en estas abstracciones:

Componente	Responsabilidad principal
`Problem`	Define el problema de optimización: fitness, longitud de genoma, construcción de genes e individuos, intensidad local, intensidad global y fitness objetivo.
`Gene`	Define la unidad atómica del genoma. Cada implementación decide cómo almacenar el valor, cómo mutarlo y cómo medir distancia a otro gen homólogo.
`Individual`	Representa una solución completa: genoma, fitness, rango, intensidad de mutación recibida y operaciones de mutación y recombinación a nivel del individuo.
`Population`	Encapsula las operaciones poblacionales: evaluación, ordenamiento por fitness, selección, recombinación y mutación.
`RankGA`	Es el *driver* de la corrida: inicializa, itera, registra salidas, decide el paro y genera resúmenes y gráficas.
`AdaptiveProblem`	Interfaz opcional para problemas cuyo estado interno puede adaptarse durante la corrida.
`ProblemFactory`	Selección explícita del problema desde la línea de comandos.

3.2. Separación entre motor y codificación

El framework no impone una sola codificación:

- puede haber genes enteros categóricos;
- genes reales de precisión doble;
- y cualquier otra representación que respete la interfaz `Gene`.

Esto tiene una consecuencia importante: la palabra *mutación* no significa lo mismo en todas las implementaciones. El motor solo calcula una **intensidad** y la pasa a cada gen. El `Gene` concreto decide qué hacer con esa intensidad.

4. Contrato formal entre problema y motor

4.1. La interfaz `Problem`

Todo problema debe proveer:

- `fitness(Individual)`: evalúa la calidad de una solución;
- `getProblemName()`: nombre visible del problema;
- `getGenomeLength()`: longitud del genoma;

- `getNewGene(boolean, Random)`: construcción de genes nuevos;
- `getNewGene(Gene)`: copia o proyección de un gen existente;
- `getGoalFt()`: fitness objetivo que, si se alcanza, detiene la corrida;
- `getDisplayModulus()`: agrupación visual del genoma en logs;
- `getNewIndividual(boolean, Random)`: construcción de individuos nuevos;
- `getNewIndividual(Individual)`: copia profunda de un individuo;
- `getLocalSearchIntensity()` y `getGlobalSearchIntensity()`.

4.2. Interpretación de las intensidades L y G

El problema define dos parámetros:

- L: intensidad local;
- G: intensidad global.

En el framework actual:

- G es la intensidad máxima asignable, típicamente asociada a los peores rangos;
- L se usa para calibrar la pendiente del perfil de mutación.

El supuesto matemático deseado es $G > L$, porque entonces el exponente de mutación resulta positivo y la mutación crece con el rango. En la implementación actual eso no se valida en tiempo de ejecución; simplemente se asume.

5. Representación de individuos y genes

5.1. Estructura de un individuo

Cada individuo contiene:

- un arreglo ordenado de genes;
- un fitness cacheado;
- un rango entero;
- la última intensidad de mutación aplicada;
- el rango de la pareja usada en la última recombinación;
- un campo textual auxiliar para logging.

La copia de un individuo es **profunda** a nivel de genes: el constructor de copia llama, para cada locus, a `problem.getNewGene(gene)`.

5.2. Importancia del contrato de copia

Este detalle es más importante de lo que parece. Si la copia de genes no se hace correctamente, la independencia estocástica entre individuos puede romperse y la interpretación experimental del algoritmo deja de ser fiable.

La lección metodológica es clara:

- el operador de copia no es un detalle administrativo;
- forma parte esencial de la semántica del algoritmo;
- una copia incorrecta puede falsear conclusiones experimentales.

5.3. Dos ejemplos de genes del repositorio

GeneInteger. Representa un gen discreto en $\{0, 1, \ldots, \texttt{NUM_VALUES} - 1\}$. Su mutación interpreta la intensidad como probabilidad de reemplazo por otro valor distinto del actual. Es decir, en este caso la intensidad sí termina siendo usada como probabilidad, pero esa decisión se toma a nivel del gen, no del motor.

GeneDoublePrecision. Representa un gen real. Su mutación aplica ruido gaussiano aditivo con desviación estándar igual a la intensidad recibida.

6. Ciclo evolutivo completo

6.1. Esquema general

Para un problema dado, una semilla, un tamaño de población N y un número de repeticiones R, el ciclo externo es:

1. crear una población inicial aleatoria;
2. evaluarla y ordenarla;
3. registrar un snapshot inicial;
4. repetir mientras no se alcance el fitness objetivo y no se haya agotado la paciencia:
   a) seleccionar;
   b) recombinar;
   c) reevaluar;
   d) registrar mejora si hubo;
   e) mutar;
   f) reevaluar;
   g) registrar mejora si hubo;
   h) adaptar el problema si implementa `AdaptiveProblem`.
5. registrar un snapshot final;
6. añadir una fila al `summary.csv`;
7. al final de todas las repeticiones, generar gráficas automáticamente.

6.2. Inicialización

La población inicial se crea con:

- tamaño fijo N;
- individuos aleatorizados;
- un `Random` derivado de la semilla base de la corrida y del índice de repetición.

Después se evalúa a toda la población y se ordena de fitness mayor a menor. El número inicial de evaluaciones de fitness es exactamente N.

7. Selección por rango

7.1. Idea

La selección no usa directamente el valor crudo del fitness. En su lugar:

1. la población ya está ordenada de mejor a peor;
2. cada individuo recibe un rango implícito por su posición;
3. ese rango determina cuántos clones se espera producir.

7.2. Presión selectiva fija

La implementación actual fija la presión selectiva en

$$S = 3.$$

No es un parámetro configurable; es una decisión de diseño del algoritmo.

7.3. Definición del rango

En la selección, el código usa

$$r_i = \frac{i}{N-1},$$

donde:

- $i = 0$ corresponde al mejor individuo;
- $i = N - 1$ corresponde al peor;
- por lo tanto $r_i \in [0, 1]$.

En la implementación que este documento describe, la definición correcta es $r_i = i/(N-1)$.

7.4. Conteo esperado de clones

El número esperado de clones del individuo i es

$$c_i = S(1 - r_i)^{S-1}.$$

Como $S = 3$, se vuelve

$$c_i = 3(1 - r_i)^2.$$

7.5. Materialización en el código

La implementación hace dos pasos:

1. asignación determinista:
$$n_i = \lfloor c_i \rfloor;$$
2. redondeo estocástico sobre la parte fraccional hasta completar exactamente N individuos.

Es decir, si
$$f_i = c_i - \lfloor c_i \rfloor,$$
entonces se hacen pasadas cíclicas y se agrega un clon extra al individuo i con probabilidad f_i, mientras el total de clones siga siendo menor que N.

7.6. Observaciones técnicas

- La selección conserva exactamente el tamaño de población.
- El procedimiento usa redondeo estocástico secuencial porque eso basta para materializar la ley de clonación adoptada por el diseño.
- El redondeo estocástico introduce un pequeño sesgo de orden por el recorrido secuencial, pero actualmente ese sesgo se acepta como parte del diseño.

8. Emparejamiento por rango y recombinación

8.1. Por qué el emparejamiento por rango es central

En este framework, el rango no solo decide cuántos clones sobreviven y qué intensidad de mutación recibe cada individuo. También decide la **vecindad reproductiva**.

Eso significa que el rango organiza simultáneamente tres mecanismos:

1. la presión selectiva;
2. la intensidad de mutación;
3. y la estructura de apareamiento.

Por eso el emparejamiento por rango debe considerarse una característica central del algoritmo y no un detalle de implementación. Si el emparejamiento fuera aleatorio, la semántica del método cambiaría de manera importante, porque se perdería la relación sistemática entre orden por fitness y mezcla genética.

8.2. Emparejamiento

La recombinación se hace en parejas adyacentes:
$$(0,1), (2,3), (4,5), \ldots$$

No hay emparejamiento aleatorio global. El criterio es puramente posicional después del ordenamiento por fitness.

8.3. Operador

El operador de recombinación a nivel de individuo es cruzamiento uniforme con **intercambio in situ**. Este detalle es esencial:

- la selección ya produjo una nueva población compuesta por clones;
- la recombinación no construye objetos "hijo" adicionales;
- en su lugar, toma dos clones ya existentes y les intercambia genes locus por locus.

Por tanto, la secuencia correcta del algoritmo es:

1. la selección multiplica genotipos mediante clonación;
2. la población clonada reemplaza a la anterior;
3. la recombinación actúa sobre esa población ya clonada;
4. cada pareja de clones se modifica directamente por intercambio de loci.

No hay una fase separada de "generación de descendencia" en el sentido clásico de crear dos hijos nuevos a partir de dos padres intactos. Los "padres" que entran a recombinación son exactamente los individuos clonados que sobreviven a la selección, y después del intercambio esos mismos objetos quedan convertidos en las soluciones recombinadas que siguen al resto del ciclo.

En cada pareja, el mecanismo es:

- para cada locus j,
- con probabilidad 0,5,
- se intercambian los genes de ambos individuos en ese locus.

Formalmente, si x_j y y_j son los genes de dos padres, entonces:

$$(x_j, y_j) \leftarrow \begin{cases} (y_j, x_j), & \text{con probabilidad } 0{,}5, \\ (x_j, y_j), & \text{con probabilidad } 0{,}5. \end{cases}$$

8.4. Consecuencias

- Los individuos cercanos en rango tienden a recombinarse entre sí.
- La mezcla genética está sesgada por la estructura del ordenamiento.
- La recombinación opera sobre clones seleccionados, no sobre una población parental separada de una población filial.
- El operador no aumenta el tamaño de la población, porque no crea individuos nuevos; solo transforma los ya existentes.
- El operador es simétrico.
- No privilegia ni prefijos ni sufijos del genoma.
- Conserva la distribución marginal de genes, aunque mezcla bloques de construcción a nivel de loci.

9. Mutación por rango

9.1. Idea central

La mutación no es plana. La intensión del diseño es:

- preservar más a los individuos bien rankeados;
- perturbar más a los individuos peor rankeados;
- hacer coexistir explotación cerca del frente y exploración en la cola de la población.

9.2. Exponente de mutación

El framework define

$$\beta = \frac{\ln(G/L)}{\ln(N-1)}.$$

Si $G > L$, entonces $\beta > 0$.

9.3. Intensidad por individuo

Para el individuo de rango i, la intensidad es

$$I_i = G\, r_i^{\beta},$$

con

$$r_i = \frac{i}{N-1}.$$

De aquí se sigue que:

- para el mejor individuo, $r_0 = 0$ y por tanto $I_0 = 0$;
- para el segundo mejor individuo, $r_1 = \frac{1}{N-1}$, y entonces

$$I_1 = G\left(\frac{1}{N-1}\right)^{\beta} = L;$$

- para el peor individuo, $r_{N-1} = 1$ y por tanto $I_{N-1} = G$;
- la intensidad crece monótonamente con el rango si $\beta > 0$.

Ese detalle no es accidental. La forma en que se define β hace que el perfil de intensidades quede anclado exactamente en tres puntos:

$$I_0 = 0, \qquad I_1 = L, \qquad I_{N-1} = G.$$

Por tanto, L no representa la intensidad del mejor individuo, sino la del primer individuo no elitista, es decir, del segundo mejor elemento de la población ordenada.

9.4. Interpretación correcta

Aquí conviene ser muy preciso. RankGA no define por sí mismo qué significa I_i. Solo define *cuánto* recibe cada individuo en una escala abstracta. El significado operacional depende del gen:

- el mejor individuo tiene intensidad exactamente 0;
- el segundo mejor individuo tiene intensidad exactamente L;
- el peor individuo tiene intensidad exactamente G.

Ese anclaje no es una interpretación opcional, sino una consecuencia directa de la forma en que el algoritmo define r_i y β.

- en genes reales, I_i puede ser desviación estándar del ruido;
- en genes categóricos, puede usarse como probabilidad de reemplazo;
- en otras codificaciones podría representar radio de vecindad o magnitud de perturbación.

Este desacoplamiento hace al framework flexible, pero también obliga a que cada problema mantenga un contrato semántico consistente.

10. Evaluación y detección de mejora

10.1. Momento de evaluación

Dentro de una iteración del ciclo, el fitness se recalcula dos veces:

1. después de recombinación;
2. después de mutación.

La selección no evalúa porque trabaja con copias de individuos cuyo fitness ya había sido conocido antes del clonaje.

10.2. Criterio de mejora

El framework actual ya no usa un único criterio de "mejora" para todo. Hay que distinguir entre:

- mejora estricta de fitness;
- reemplazo del incumbente;
- y reinicio de paciencia.

En todos los casos se exige además que el nuevo candidato no sea genotípicamente idéntico al incumbente previo. En código, la distancia cuadrática entre ambos debe ser positiva.

Modo `strict`. Si la política de incumbente es `strict`, entonces el reemplazo del incumbente exige:

- el fitness debe ser estrictamente mayor;
- la distancia cuadrática entre genomas debe ser positiva.

Modo `neutral`. Si la política de incumbente es `neutral`, entonces también se permite sustituir al incumbente cuando:

- el fitness es igual al del incumbente actual;
- pero la distancia cuadrática entre genomas sigue siendo positiva.

Es decir, en modo `neutral` sí se permite que el representante del mejor fitness alcanzado se desplace sobre una meseta neutral. Lo que no se permite en ningún caso es reemplazarlo por el mismo genotipo.

10.3. Distancia entre individuos

La distancia entre individuos se calcula como

$$d^2(x, y) = \sum_{j=1}^{n} \left(d_j(x_j, y_j)\right)^2 ,$$

donde d_j es la distancia definida por el gen en el locus j.

En genes enteros tipo Hamming unilocus, cada término vale 0 o 1. En genes reales, depende de la implementación concreta del gen.

11. Adaptación opcional del problema

Si un problema implementa `AdaptiveProblem`, entonces el driver llama:

```
adapt(bestFitness)
```

después de completar la iteración evolutiva.

Esto permite que el problema:

- ajuste pesos internos;
- cambie parámetros de representación;
- modifique penalizaciones;
- o altere alguna heurística dependiente del progreso.

Si el problema no implementa esa interfaz, no ocurre nada. La adaptación no forma parte del contrato básico de `Problem`.

12. Criterios de paro

12.1. Paro por objetivo

La corrida termina si el mejor fitness actual satisface

$$f_{\text{máx}} \geq f_{\text{goal}},$$

donde $f_{\text{goal}} =$ `problem.getGoalFt()`.

12.2. Paro por paciencia

También termina si transcurre un tiempo fijo sin el tipo de progreso que la política de paciencia considere suficiente.

La versión actual permite parametrizar dos cosas distintas:

- el criterio de reemplazo del incumbente;
- y el criterio de reinicio de paciencia.

La paciencia se configura con un parámetro en milisegundos:

$$\texttt{patienceMillis}.$$

Por defecto:

$$\texttt{patienceMillis} = 60\,000 \text{ ms} = 1 \text{ minuto}.$$

12.3. Política de incumbente

El framework distingue dos modos:

- `strict`: el incumbente solo se sustituye si aparece un individuo con fitness estrictamente mayor;
- `neutral`: el incumbente también puede sustituirse por otro genotipo con el mismo fitness, siempre que la distancia genotípica sea positiva.

Esto controla si el representante del mejor fitness alcanzado puede o no desplazarse sobre una meseta neutral.

12.4. Política de reinicio de paciencia

El reinicio de paciencia también es configurable:

- `fitness`: la paciencia se reinicia solo con mejora estricta de fitness;
- `movement`: la paciencia se reinicia cuando cambia el incumbente según la política de incumbente seleccionada.

La distinción es importante. Permitir movimiento neutral del incumbente no obliga a interpretar ese movimiento como progreso suficiente para seguir extendiendo la corrida. Son dos decisiones metodológicas distintas.

12.5. Consecuencia importante

La paciencia es **por repetición**, no por experimento completo. Si se ejecutan 100 repeticiones y ninguna alcanza el objetivo, el tiempo total puede escalar aproximadamente a 100 minutos, salvo pequeñas variaciones por inicialización, I/O y sistema operativo.

13. Contabilidad de evaluaciones

13.1. Qué se cuenta

La variable `evaluations` cuenta cuántas veces se evaluó fitness en una repetición.

Se suma:

- N al inicializar;
- N después de recombinación en cada generación;
- N después de mutación en cada generación.

13.2. Corrección importante ya incorporada

La versión actual ya no fuerza una generación extra si la población inicial alcanza el objetivo. El ciclo principal usa un `while` y no un `do-while`. Eso evita sobrecontar evaluaciones en casos donde la solución objetivo ya aparece desde el principio.

14. Salida experimental

14.1. Logs heredados

Por cada corrida se generan:

- un archivo `.txt` con snapshots cronológicos del mejor individuo;
- un archivo `_rep.txt` por repetición con el último estado de la población en esa repetición.

14.2. Resumen estructurado

También se genera un `summary.csv` con una fila por repetición. Entre sus columnas están:

- algoritmo;
- clase del problema;
- nombre del problema;
- parámetros efectivos del problema;
- semilla;
- `run_id`;
- índice de repetición;
- tamaño de población;
- número total de repeticiones;
- número de evaluaciones;
- mejor fitness final;
- tiempo transcurrido en milisegundos;
- razón de terminación;
- fitness objetivo;
- paciencia configurada en milisegundos;
- política de reemplazo del incumbente;

- política de reinicio de paciencia;
- prefijo del archivo de salida.

14.3. Gráficas automáticas

Al terminar todas las repeticiones, RankGA invoca un script en Python que genera:

- una gráfica ordenada por evaluaciones;
- una gráfica ordenada por tiempo transcurrido;
- y dos CSV ordenados correspondientes.

La versión actual ya incorpora el `runId` en el nombre de la figura para evitar colisiones entre corridas con el mismo problema y la misma semilla.

15. Ejemplo: OneMax de 8 bits

15.1. Definición

En el problema OneMax de longitud $n = 8$:

- el genoma es binario;
- el fitness es la suma de bits en uno;
- el objetivo es alcanzar fitness 8.

Formalmente, si $x \in \{0, 1\}^8$, entonces

$$f(x) = \sum_{j=1}^{8} x_j.$$

15.2. Configuración en el repositorio

La implementación actual:

- usa `GeneInteger` con dominio binario;
- define intensidad local $L = 1/8$;
- define intensidad global $G = 0{,}5$;
- usa una estrategia de copia de genes que preserva la independencia estocástica entre individuos.

15.3. Interpretación experimental

OneMax es un benchmark deliberadamente sencillo. Si este problema falla con frecuencia por paciencia, eso suele indicar:

- un defecto en la implementación;
- una semántica inconsistente de mutación;
- o un error en el diseño experimental y no una limitación genuina del algoritmo.

16. Supuestos y limitaciones del framework

16.1. Supuestos fuertes

El algoritmo presupone:

- fitness orientado a maximización;
- poblaciones de tamaño al menos 3;
- $G > L$ para que la mutación dependa del rango de forma monótona;
- una implementación consistente del operador de copia de genes;
- y una interpretación bien definida de intensidad dentro de cada tipo de gen.

16.2. Decisiones de diseño

- La presión selectiva S es fija.
- El emparejamiento para recombinación es adyacente y no aleatorio.
- La selección usa redondeo estocástico secuencial y no necesita muestreo multinomial exacto para cumplir su objetivo.

16.3. Limitaciones reales de implementación

- El motor usa estado estático en `RankGA`, por lo que está diseñado como *runner* simple, no como motor concurrente para varias corridas simultáneas en la misma JVM.
- La generación automática de gráficas depende de componentes externos disponibles en el entorno.

16.4. Criterio documental

Para entender el comportamiento real del framework, hay que preferir:

1. el código ejecutable;
2. la ejecución efectiva de los tests automáticos del proyecto, cuando son aplicables al cambio o a la afirmación que se quiere sostener;
3. las corridas reproducibles y sus resúmenes experimentales estructurados;
4. y la revisión manual de archivos de salida cuando hace falta una validación complementaria.

16.5. Observación práctica sobre verificación

En este repositorio conviene distinguir entre:

- compilación correcta del código;
- ejecución efectiva de pruebas automáticas;
- y validación manual mediante corridas, inspección de logs, `summary.csv` y gráficas.

Desde el punto de vista metodológico, esa distinción importa porque una afirmación del tipo "esto ya está probado" no significa lo mismo si se basa en una batería automatizada completa, en una corrida reproducible o en una inspección manual del *summary*. En una descripción rigurosa del framework, esas fuentes de evidencia no deben confundirse ni presentarse como si tuvieran exactamente el mismo alcance.

17. Resumen conceptual final

RankGA puede resumirse como sigue:

1. ordena la población por fitness;
2. replica individuos según su rango, no según el valor crudo del fitness;
3. cruza por parejas adyacentes;
4. muta con intensidad creciente conforme empeora el rango;
5. preserva al mejor individuo porque su intensidad es cero;
6. asigna al segundo mejor individuo exactamente la intensidad local L;
7. permite adaptación opcional del problema;
8. y produce salidas estructuradas para análisis experimental.

La contribución práctica del framework no es solo combinar selección por rango con mutación dependiente del rango, sino hacerlo de manera genérica para varias codificaciones y problemas. El precio de esa generalidad es que el contrato entre motor y genes debe estar especialmente bien cuidado. Cuando ese contrato se respeta, el comportamiento es razonable y reproducible; cuando se viola, incluso problemas muy fáciles pueden dar resultados engañosos.